


```
marp: true
theme: default
paginate: true
size: 16:9
backgroundColor: '#0B1220'
color: '#E2E8F0'
style: |
/* NotebookLM-pattern dark theme: deep navy background,
orange primary
accent (#F97316), green secondary accent (#10B981) for
success metrics
and outcome labels. Mirrors the visual language of
talk/Portable_Agent_Context.pdf (NotebookLM design reference).
/
```

The agent is new here every time

Most coding agents enter a repo as a bright visitor with no durable map.

- They rediscover the tree.
- They infer ownership from whatever file they open first.
- They miss the invariant that decides whether an answer is safe.
- They repeat the same exploration in the next session, editor, or model.

A **cold session** — unknown files, unclear risks, expensive search — gets converted into **bounded work** — read order, named blast radius, verified output — only when the repo carries its own map.

A warm start for every coding agent

The repo carries its own map, routing rules, and verification shortcuts.

The failure is often not intelligence. It is missing repo evidence.

We already check in tests, docs, schemas, CI, runbooks, and migration history.

Agent context should be another checked-in repo artifact: the map of what is true, risky, connected, and out-of-bounds.

Portable	Reviewable	Fresh
Markdown / JSON, not vendor memory.	Lives in git. Reviewed like code changes.	Drift is a failing preflight, not a quiet footnote.

The standard is simple: read the repo map before editing the repo.

Every serious corpus needs a navigation layer

The dual-routing architecture generalizes to any file system or stable corpus an agent must explore before acting:

1. Stable datasets
2. Design systems
3. Operations runbooks
4. **Code repositories** ← we focus here today

But today, we focus on where it hurts developers the most: code repositories.

Explorable repo recall as a three-track system

agent-context works when **navigation**, **operating loop**, and **engineering** are treated as separate controls.

Explorable recall as a three-track system

agent-context works when navigation, consumption, and maintenance are treated as separate controls.

Navigation

CONTROL SURFACE

What should load,
what should not,
and when should it stop?
read order / risky paths

Operating loop

AGENT BEHAVIOR

How does this agent
consume repo context
during real work?
trust route / verify route

Engineering

SYSTEMS LAYER

What makes prior work
durable, inspectable,
and reusable?
manifest / tests / CI

agent context outcomes

Today's session: the Navigation track

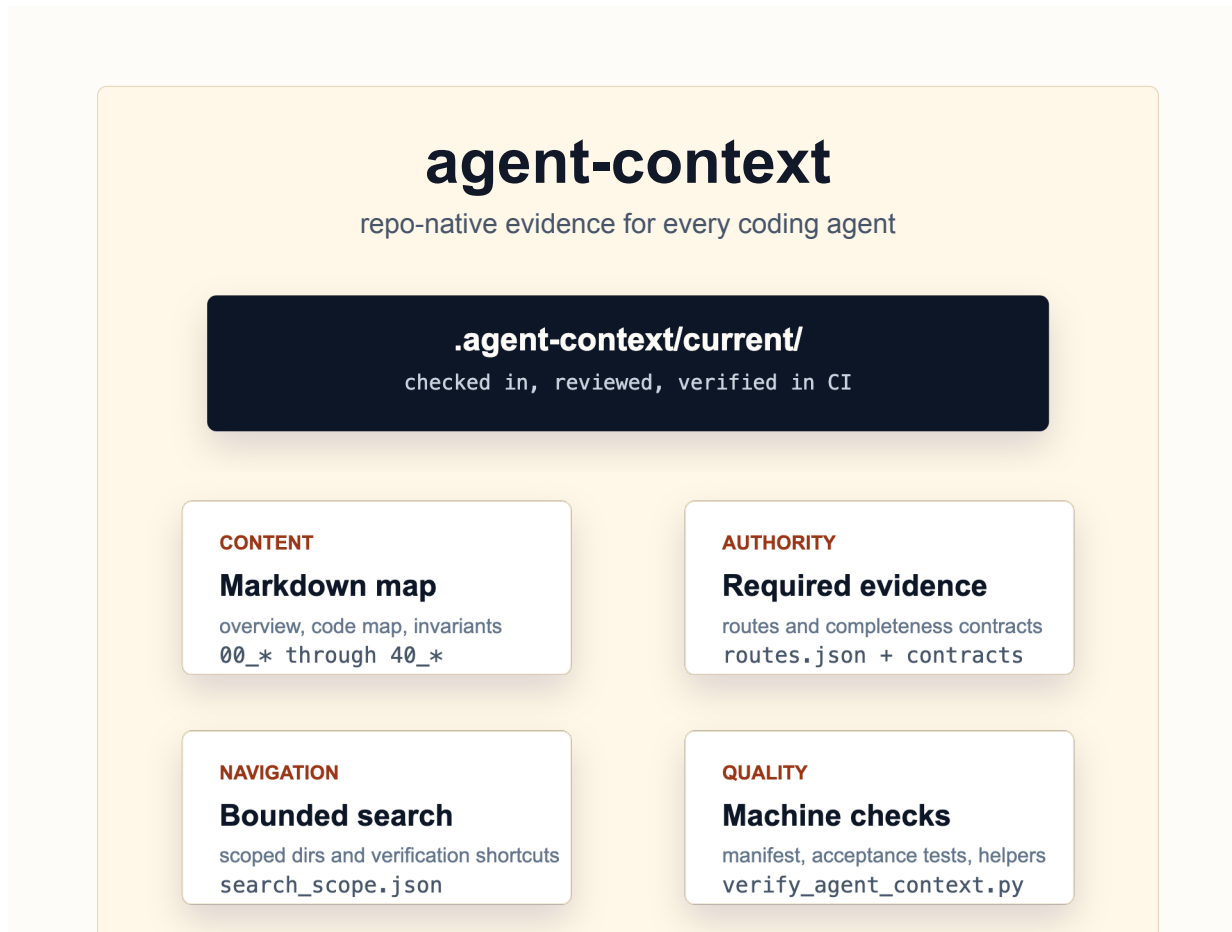
The three tracks are companions, but each deserves its own focused conversation. **Today is the Navigation track** — the control surface that decides:

- **What to load** — completeness contracts that name every file family that must change together.
- **What not to load** — search scopes that bound where the agent grep-walks.
- **When to stop** — stop rules and verification shortcuts that say "you have enough to answer."

Operating Loop (how Claude vs Cursor vs Codex actually *consume* the same pack) and Engineering (manifest / tests / CI / freshness gates) are deliberately out of scope for this hour — covered through the methodology and tradeoff slides at a depth that supports the navigation story, not as standalone deep dives.

The artifact: `.agent-context/`

A committed folder next to the code. No hosted service. No hidden memory. No editor lock-in.



Same navigation design, opposite agent loops

The pack is one thing. The reading patterns it triggers are two:

Trust-and-Follow (Claude / Gemini)	Search-and-Verify (Cursor / Codex)
Reads completeness contracts. Stops when the contract says done.	Bounds its grep to scoped directories and cross-checks verification shortcuts.
Compressed search → fewer files opened.	Expanded proof burden → more verification, fewer dead ends.
14 → 4 files opened (claude bare → structured).	3 → 12 files opened (cursor — but each carries a verified-against-shortcut signal).

Model-agnostic by construction. The routing block (`CLAUDE.md` / `AGENTS.md` / `.cursorrules`) points tools at `.agent-context/` before source exploration.

The engineering pipeline (a repeatable loop)

Not a library trick. An automated, checkable pipeline.

1. **Initialize** — scaffold the pack.
2. **Fill** — agent enumerates subsystems, fills templates (driven by `SKILL.md`).
3. **Verify** — structural checks, schema validation, grep-backed acceptance tests.
4. **Freshness** — fail the run when relevant code changed but context did not.
5. **Measure** — compare bare vs `structured_fresh` runs on hard tasks.

```
agent-context init . && agent-context verify . && agent-context freshness .
```

That is the meetup takeaway: **a repeatable operating loop, not a magic prompt.**

What a reviewer actually checks

The review target is the context **diff**, not the model's private memory.

- **File families:** does the pack name every file family that must change together?
- **Negative guidance:** does it say which plausible files are deprecated, generated, or near-misses?
- **Silent failures:** does it call out the thing that can break without a compile error?

Every agent starts from the same repo evidence

The routing block points tools at `.agent-context` before source exploration.



Claude Code

```
Read 00_START_HERE.md first
Load invariants and code map
Open only named source paths
```



Codex CLI

```
Read routes.json + search_scope.json
Search only relevant directories
Verify shortcut signatures
```

How to test whether it works

The methodology is deliberately boring:

Protocol piece	Why it matters
<code>bare clone</code> vs <code>structured_fresh clone</code>	Same repo, same task, only context changes.
Hard multi-hop tasks	Lookup tasks prove little; synthesis exposes wrong turns.
Grep-backed ground truth	Every expected file family is mechanically checkable.
Independent judge + audit	Self-scores are evidence, not verdicts.

Freshness is a hard experiment gate. If the pack is stale, the run fails before agents start.

Quantified evidence — Q2 2026 multi-agent rerun

252 graded answers · 48 cells · 6 repos · 4 model variants · bare vs structured_fresh.

Fresh-pack isolated protocol; LLM-provisional grading via independent Claude Code subagents (one per cell, fresh context).

Agent / Model	Bare yes-rate	Structured yes-rate	Δ
Claude Opus 4.7 (trust-and-follow)	80% (4.80/6)	100% (6.00/6)	+20pp
Cursor <code>claude-opus-4-7-medium</code>	89% (5.33/6)	97% (5.83/6)	+8pp
Cursor <code>composer-2-fast</code>	61% (3.67/6)	81% (4.83/6)	+20pp
Codex CLI 0.130.0	72% (4.33/6)	78% (4.67/6)	+6pp

- Claude Opus + structured pack: 6/6 perfect across all 6 repos.
- Cursor `composer-2-fast` (the default): biggest absolute correctness lift — +20pp

What each agent's strongest signal looks like

Different agents expose different telemetry. Each lane is reported on its strongest measured signal — no blended scoreboard.

Agent	Strongest measured signal	Bare	Structured
Claude (trust-and-follow)	Files opened / task (3-repo historical)	6.3	1.9
Claude (trust-and-follow)	Tokens / task (3-repo historical)	38.6K	13.1K
Cursor composer-2-fast (search-and-verify)	Yes-rate (Q2 rerun, 6 repos)	61%	81%
Cursor claude-opus-4-7-	Median duration / task (Q2	219s	78s

Tradeoffs, not magic

Tradeoff	What it means
Maintenance cost	The pack must change when relevant code changes.
Overfitting risk	Packs should route and bound search, not quote every answer.
Repo-size fit	Small repos may only need tier 1. Full packs scale to complex/monorepo.
Telemetry gaps	Cursor stores sessions in opaque SQLite; tokens/cost stay null without reverse-engineering.
Human review remains	Context improves first drafts; it does not replace code review.

The bargain: spend a little repo-maintenance effort to stop every agent from rediscovering the same map.

Try this on one repo tomorrow

That is enough to decide whether the repo needs the full tier 3 pack.

1. **Pick one painful workflow** — impact analysis, auth, cache, release, migration, or docs corpus.
2. **Create tier 1** — start with code map + search scope.
3. **Add one invariant** — name the silent failure and near-miss files.
4. **Run bare vs context** — same task, same repo, two clones.
5. **Review the diff** — files opened, dead ends, missing surfaces, risk.

```
agent-context init . && agent-context verify . && agent-context freshness .
```


Make repo knowledge explicit

Install the skill into your agent of choice (one-time):

```
# Install for Claude Code:  
git clone https://github.com/cote-star/agent-context.git \  
  && cp -r agent-context/skills/agent-context ~/.claude/skills/  
  
# Cursor reads .cursorrules natively. Open repo, no install needed.
```

Then in any repo, ask your agent:

Set up agent context for this repo.

Start small with **tier 1**. Promote to **tier 3** when the repo has cross-cutting invariants or production-risk workflows. Context is a checked-in interface between repos and agents.

Make context part of the repo

Not a prompt trick. Not a vendor feature. A checked-in standard teams can review.

`init` · `verify` · `freshness` · `bare vs structured_fresh`

Today: **Navigation track**. Operating Loop and Engineering tracks are companion conversations.

github.com/cote-star/agent-context

— *Amit Prusty*